XRPL Validator — Daily Engineering Report · 2026-09-03

Period: 2026-09-03 00:00 to 24:00 US Eastern (2026-09-03 04:00 to 2026-09-04 04:00 UTC); ledgers #106,725,811 to #106,748,087. Every count from the ledger and the validator's logs stops at #106,748,087. Generated 2026-10-21 00:47 EDT by report generator v3.0. Sources: the XRP Ledger (ledger headers and amendments, read from a reference rippled node), the validator's signing log and receipt logs, the deployment pipeline's gate, soak and health records, the git history of xrpl-validator and the XRPL Rust SDK, and data.xrpl.org. Scope: XRPL work only.

Revised on 2026-10-21 in report format v3; the original is kept in this repository's history. Built only from records that exist for this day. Available: the day's first and last ledger, from a public full-history server; Step 1 receipt records (they begin 2026-08-29); xrpl-validator merge records (they begin 2026-07-04); data.xrpl.org agreement scores. Not available: ledger headers (the reference node holds ledgers from 2026-10-04 22:52 UTC); signing records (they begin 2026-09-19); Steps 2–5 receipt records (they begin 2026-10-09); deployment records (they begin 2026-09-11); amendment data.

Status

Not assessed — no signing records for this day.

Criteria. Normal: coverage outside planned maintenance at or above the 99.9% objective, every validation agreed with the network's validated ledger, and no engine receipt is open. Degraded: coverage below the objective, or an open engine receipt. Impaired: any validation that disagreed with the network, or coverage below 90%.

Executive summary

- **Validator:** no signing records for this day (they begin 2026-09-19).
- **Native engine:** 101 engine receipts recorded (56 fixed, 0 explained, 45 without a recorded fix).
- **Changes:** 0 deployments, 55 commits merged into main.

Service levels

No signing records for this day (they begin 2026-09-19).

External agreement, data.xrpl.org (UTC day 2026-09-03): 0.79892 (186 missed of 925). *data.xrpl.org scores UTC days from the validations its crawlers received.*

Native engine conformance

Ledgers with a receipt in the period (#106,725,811–#106,748,087), by step:

Step	Ledgers with a receipt
Step 1 · native engine	101
Step 2 · records	0
Step 3 · order	0
Step 4 · peer data	0
Step 4 · validations	0
Step 5 · own hash	0
Writer	0

Live counters are kept only for the current run; none exist for a past day.

Receipt register

ID	Time	Ledger(s)	Step	Description	Disposition
R-01	—	#106,727,063	Step 1 · native engine	2 object(s) differed in content	no disposition recorded
R-02	—	#106,727,070	Step 1 · native engine	1 object(s) differed in content	no disposition recorded
R-03	—	#106,727,083	Step 1 · native engine	4 object(s) differed in content	no disposition recorded
R-04	—	#106,727,096	Step 1 · native engine	1 ledger change(s) the network made were absent from ours: 0339209F; 1 change(s) of ours the network did not make; 4 object(s) differed in content	fixed: commit 9534ff1, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-05	—	#106,727,098	Step 1 · native engine	3 object(s) differed in content	no disposition recorded
R-06	—	#106,727,665	Step 1 · native engine	1 object(s) differed in content	no disposition recorded
R-07	—	#106,730,304	Step 1 · native engine	1 object(s) differed in content	fixed: commit 029a608, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-08	—	#106,730,661	Step 1 · native engine	2 object(s) differed in content	fixed: commit f55ce32, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-09	—	#106,731,077	Step 1 · native engine	2 object(s) differed in content	no disposition recorded
R-10	—	#106,731,793	Step 1 · native engine	6 object(s) differed in content	fixed: commit f7156c9, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-11	—	#106,732,165	Step 1 · native engine	1 object(s) differed in content	fixed: commit f852b6b, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-12	—	#106,732,759	Step 1 · native engine	6 object(s) differed in content	fixed: commit 44348d2, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-13	—	#106,732,842	Step 1 · native	6 ledger change(s) the network made were absent from ours: CF6BC0D4, 97769BF5 (deleted), C8CE3EBC; 2 object(s) differed	fixed: commit 67140df, deployed in cycle 121

ID	Time	Ledger(s)	Step	Description	Disposition
			engine	in content	(2026-09-11 17:39 UTC)
R-14	—	#106,732,893	Step 1 · native engine	1 object(s) differed in content	fixed: commit 66db675, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-15	—	#106,733,048	Step 1 · native engine	1 object(s) differed in content	fixed: commit f852b6b, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-16	—	#106,733,194	Step 1 · native engine	1 object(s) differed in content	no disposition recorded
R-17	—	#106,733,197	Step 1 · native engine	6 ledger change(s) the network made were absent from ours: 54123B87 (deleted), 119270E3 (deleted), 54123B87 (deleted); 4 object(s) differed in content	fixed: commit b0e848e, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-18	—	#106,733,288	Step 1 · native engine	4 change(s) of ours the network did not make; 1 object(s) differed in content	fixed: commit e9f2d58, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-19	—	#106,733,664	Step 1 · native engine	1 object(s) differed in content	fixed: commit b6f8c45, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-20	—	#106,734,110	Step 1 · native engine	1 object(s) differed in content	fixed: commit 66db675, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-21	—	#106,734,208	Step 1 · native engine	1 object(s) differed in content	fixed: commit 4199029, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-22	—	#106,734,370	Step 1 · native engine	5 object(s) differed in content	fixed: commit e9f2d58, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-23	—	#106,734,469	Step 1 · native engine	6 object(s) differed in content	no disposition recorded
R-24	—	#106,734,485	Step 1 · native engine	2 object(s) differed in content	fixed: commit ad7b3eb, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-25	—	#106,734,622	Step 1 · native engine	1 object(s) differed in content	fixed: commit 4199029, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-26	—	#106,734,683	Step 1 · native engine	2 ledger change(s) the network made were absent from ours: A76D0723, 063E84DB; 3 object(s) differed in content	fixed: commit e9f2d58, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-27	—	#106,734,972	Step 1 · native engine	2 object(s) differed in content	fixed: commit 4199029, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-28	—	#106,735,332	Step 1 · native engine	4 ledger change(s) the network made were absent from ours: C0148750, 2DE7C6A0, 9B7CECDD; 3 change(s) of ours the network did not make; 2 object(s) differed in content	fixed: commit fdb2592, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-29	—	#106,735,554	Step 1 · native engine	1 object(s) differed in content	fixed: commit b6f8c45, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-30	—	#106,735,594	Step 1 · native engine	3 object(s) differed in content	fixed: commit 6abd978, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-31	—	#106,735,617	Step 1 · native engine	3 object(s) differed in content	no disposition recorded

ID	Time	Ledger(s)	Step	Description	Disposition
R-32	—	#106,735,862	Step 1 · native engine	2 change(s) of ours the network did not make; 3 object(s) differed in content	no disposition recorded
R-33	—	#106,735,879	Step 1 · native engine	2 ledger change(s) the network made were absent from ours: 37AAC93D (deleted), A5D484CE (deleted); 2 object(s) differed in content	no disposition recorded
R-34	—	#106,735,893	Step 1 · native engine	4 object(s) differed in content	no disposition recorded
R-35	—	#106,735,894	Step 1 · native engine	2 ledger change(s) the network made were absent from ours: 37AAC93D (deleted), 838C9B51 (deleted); 2 object(s) differed in content	no disposition recorded
R-36	—	#106,735,895	Step 1 · native engine	2 transaction result(s) differed from rippled's; 9 ledger change(s) the network made were absent from ours: E03D7ED4, 2F5BD2C6, B8BCE8D0; 1 change(s) of ours the network did not make; 16 object(s) differed in content	no disposition recorded
R-37	—	#106,735,896	Step 1 · native engine	1 object(s) differed in content	no disposition recorded
R-38	—	#106,735,916	Step 1 · native engine	2 ledger change(s) the network made were absent from ours: 1B7B8BF6 (deleted), 37AAC93D (deleted); 2 object(s) differed in content	no disposition recorded
R-39	—	#106,735,981	Step 1 · native engine	8 object(s) differed in content	no disposition recorded
R-40	—	#106,735,983	Step 1 · native engine	2 change(s) of ours the network did not make; 6 object(s) differed in content	fixed: commit 8dca547, deployed in cycle 121 (2026-09-11 17:39 UTC)

A receipt is recorded whenever the validator's own work differs from the network's, or the network's trusted validations reached it short of quorum. One ledger flagged by several steps is one receipt. Dispositions: fixed (with the finding, commit and deployment), explained (not a defect, with the reason), or open. Receipt and incident IDs are numbered per report: R-01 here is R-2026-09-03-01, INC-01 is INC-2026-09-03-01. Times are not shown where the reference node no longer holds the ledger.

61 more receipts (#106,735,988–#106,747,838) are in the record and not listed individually; their dispositions: 33 fixed, 28 no disposition recorded.

Incidents

Not assessed: no signing records for this day.

Change log

Deployments

No deployment records before 2026-09-11.

Merged into main

Merged	Repo	Commit	Subject
05:22 UTC	xrpl-validator	`32512b0`	fix(ledger): single-path bridge legs take and judge the pool offer rippled's tip() hands back (findings 103, 111)
06:22 UTC	xrpl-validator	`9534ff1`	fix(ledger): a dust offer whose quality rounds worse is removed, not crossed (finding 112)
09:16 UTC	xrpl-validator	`029a608`	fix(ledger): erasing a sender's in-flight line removes its entries, not the pages (finding 113)
10:33 UTC	xrpl-validator	`f55ce32`	fix(ledger): killed crossings still reap what the rev pass stepped over (finding 114)
11:26 UTC	xrpl-validator	`f7156c9`	fix(ledger): the rev pass steps past a maker it emptied and reaps behind it (finding 115)
12:05 UTC	xrpl-validator	`67140df`	fix(ledger): judge the taker's limit by the level's quality, not the offer's own rate (finding 116)
12:15 UTC	xrpl-validator	`b0e848e`	fix(ledger): a level this apply emptied is stepped through, not reported live (finding 117)
13:13 UTC	xrpl-validator	`44348d2`	fix(ledger): the first book hop composes its in-side transfer rate into the strand's quality function (finding 119)
13:13 UTC	xrpl-validator	`f852b6b`	fix(ledger): the remaining-in budget is an STAmount (finding 118)
14:00 UTC	xrpl-validator	`e9f2d58`	fix(ledger): the pool turn anchors on the residual tip inside the inflated limit — the taker's own tip included (findings 120, 121)
14:45 UTC	xrpl-validator	`4199029`	fix(ledger): the funds-exhausting fill takes the line's own balance; a bridged round's multiPath and self-tip removal follow rippled's pass (findings 122, 123; finding 121 refined)
14:53 UTC	xrpl-validator	`6abd978`	fix(ledger): the out-limited input is clamped to the underfunded maker's own input (finding 124)
15:37 UTC	xrpl-validator	`fdb2592`	fix(ledger): a bridged round ranks the direct strand on the book's live tip, not the emptied level under the cursor (finding 125)
15:37 UTC	xrpl-validator	`b6f8c45`	fix(ledger): the mixed engine's book segment nets its carry at mulRatio-nearest (finding 126)
16:02 UTC	xrpl-validator	`e7b7927`	fix(ledger): an XRP side of a maker's residual subtracts in exact drops (finding 127)
16:15 UTC	xrpl-validator	`8dca547`	fix(ledger): a bridged strand's limit-out function composes the tip's offer, not the forced pool (finding 128)
16:25 UTC	xrpl-validator	`66db675`	fix(ledger): an encoded rate rounds to nearest after the legacy half-up digit; the fib seed multiplies at nearest (findings 129, 130)
17:18 UTC	xrpl-validator	`fc91ef2`	fix(ledger): a DepositPreauth entry carries Flags and its owner-directory node (finding 132)
17:18 UTC	xrpl-validator	`ad7b3eb`	fix(ledger): a forward-driven pool turn swaps the carried input and clamps only the delivery (finding 131)
19:30 UTC	xrpl-validator	`7a5e5f4`	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)
19:31 UTC	xrpl-validator	`5879b9c`	fix(ledger): a book pass takes the whole same-quality group; pool slices price by their 16-digit rate (findings 137, 138)
19:31 UTC	xrpl-validator	`0fd2a90`	fix(ledger): SignerListSet replace re-inserts the owner-directory entry (finding 136)
19:31 UTC	xrpl-validator	`c15f3b6`	fix(ledger): reserve gate applies to crossed offers on the pre-fee balance (finding 135)
19:31 UTC	xrpl-validator	`be1a76d`	fix(ledger): the taker's spend is capped by the deferred-credits balance; a maker's XRP reserve holds at the flow's original owner count (findings 133, 134)
19:32 UTC	xrpl-validator	`4d86d53`	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)

Merged	Repo	Commit	Subject
19:33 UTC	xrpl-validator	`0ff3de8`	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)
19:33 UTC	xrpl-validator	`81f8f89`	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)
19:33 UTC	xrpl-validator	`cb0ade1`	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)
19:33 UTC	xrpl-validator	`98ba356`	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)
19:33 UTC	xrpl-validator	`c6deec0`	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)
19:33 UTC	xrpl-validator	`3dac6cd`	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)
19:33 UTC	xrpl-validator	`52b34c3`	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)
19:33 UTC	xrpl-validator	`a73e77b`	fix(ledger): a book pass takes the whole same-quality group; pool slices price by their 16-digit rate (findings 137, 138)
19:33 UTC	xrpl-validator	`c266cff`	fix(ledger): SignerListSet replace re-inserts the owner-directory entry (finding 136)
19:33 UTC	xrpl-validator	`4eed25f`	fix(ledger): reserve gate applies to crossed offers on the pre-fee balance (finding 135)
19:33 UTC	xrpl-validator	`1265494`	fix(ledger): the taker's spend is capped by the deferred-credits balance; a maker's XRP reserve holds at the flow's original owner count (findings 133, 134)
19:34 UTC	xrpl-validator	`75efdb1`	fix(ledger): a book pass takes the whole same-quality group; pool slices price by their 16-digit rate (findings 137, 138)
19:48 UTC	xrpl-validator	`c8774b1`	fix(ledger): a book pass takes the whole same-quality group; pool slices price by their 16-digit rate (findings 137, 138)
19:48 UTC	xrpl-validator	`f6c5d75`	fix(ledger): SignerListSet replace re-inserts the owner-directory entry (finding 136)
19:48 UTC	xrpl-validator	`e7b4239`	fix(ledger): reserve gate applies to crossed offers on the pre-fee balance (finding 135)
19:48 UTC	xrpl-validator	`b28172a`	fix(ledger): the taker's spend is capped by the deferred-credits balance; a maker's XRP reserve holds at the flow's original owner count (findings 133, 134)
19:54 UTC	xrpl-validator	`168f252`	fix(ledger): strands are admitted against the fee-inclusive limit; self-offers leave the book only when the direct strand executes (finding 139)
19:56 UTC	xrpl-validator	`2399f83`	fix(ledger): a path that only names the SendMax issuer at its head is the default strand (finding 140)
20:09 UTC	xrpl-validator	`a4080bb`	fix(ledger): an unanchored pool tips its leg by SPOT, and the single pass is sized to the limit (finding 141)
21:35 UTC	xrpl-validator	`03e305a`	fix(ledger): iteration 0 takes multiPath from the strands activateNext keeps (finding 142)
22:12 UTC	xrpl-validator	`2155052`	fix(ledger): the crossing's residual is the original less the STAmount sum of the iterations' fills (finding 144)
22:12 UTC	xrpl-validator	`6e3466f`	fix(ledger): the direct pool is consulted in a single-path round only if its strand survived activation (finding 145)
22:47 UTC	xrpl-validator	`47905e9`	fix(ledger): a strand's first pool hop is sized by the reverse pass, not swept to the SendMax (finding 147)
00:16 UTC	xrpl-validator	`e801434`	fix(ledger): re-run a limiting book level and re-price its last maker (finding 149)
00:21 UTC	xrpl-validator	`bafc30c`	fix(ledger): record a credential's directory pages (finding 150)

Merged	Repo	Commit	Subject
01:11 UTC	xrpl-validator	`802b6a7`	fix(ledger): a strand that flows nothing never returns (finding 151)
01:11 UTC	xrpl-validator	`02653b5`	fix(ledger): an expired check cannot be cashed or created (finding 152)
02:26 UTC	xrpl-validator	`399e5b4`	fix(ledger): the direct pass removes a self-offer only inside the limit, and stops at one beyond it (finding 143)
02:41 UTC	xrpl-validator	`0bbd06e`	test(node): bundles rebuild in-ledger objects exactly and can pin untouched targets
03:33 UTC	xrpl-validator	`18ff76e`	fix(ledger): an offer that became unfunded during the crossing is not removed for good (finding 153)

Findings fixed

No engine findings merged into main this day.

Other engineering changes

- **fix(ledger): single-path bridge legs take and judge the pool offer rippled's tip() hands back (findings 103, 111)** (`32512b0` , xrpl-validator) rippled's BookStep runs tryAMM(tip) before the tip's own offer. In single-path mode getOffer(tip) anchors a pool offer at the tip's quality (changeSpotPriceQuality) whenever the pool's spot beats the tip by more than 1e-7 — or hands back maxOffer when the tip sits beyond the limit — and that offer is consumed FIRST; the tip's own level follows on a later pass (execOffer breaks on the quality change). tip() judges …
- **fix(ledger): a dust offer whose quality rounds worse is removed, not crossed (finding 112)** (`9534ff1` , xrpl-validator) rippled's OfferStream::step (fixRmSmallIncreasedQOffers) drops an offer whose effective amounts — clipped to the owner's funds with ceilOutStrict when it is not the issuer and short — have a TakerPays at the asset's smallest positive unit (one drop) and a quality, recomputed from those amounts, worse than the directory's; TakerGets in XRP is never subject to it, and two IOUs only when TakerPays < TakerGets. The …
- **fix(ledger): erasing a sender's in-flight line removes its entries, not the pages (finding 113)** (`029a608` , xrpl-validator) Intermediate acquisitions ride through the SENDER in our strand walk: a hop credits the sender's line in the hop currency and the next hop debits it, and the net-zero line drops out of the mutation set. When that line did not exist, its creation also inserted the sender's key into two owner directories, and the walk erased those traces by restoring the sender's line, both directory roots and both LAST pages from …
- **fix(ledger): killed crossings still reap what the rev pass stepped over (finding 114)** (`f55ce32` , xrpl-validator) #106730661 3241437EB11D — a tfImmediateOrCancel buy of 20.575959 XRP for 28.215814 RLUSD by a taker holding 0.0000206 RLUSD. Mainnet: tecKILLED, nothing crossed, and the two EXPIRED offers 729F17CE / 392EFC07 on the level behind the tip are deleted with their page. Ours: tecKILLED, nothing deleted (differential 84/85).
- **fix(ledger): the rev pass steps past a maker it emptied and reaps behind it (finding 115)** (`f7156c9` , xrpl-validator) #106731793 E3E8726FA355 — rGodbj1g sells 1.5 XRP for XRPCAT (tfSell|tfImmediateOrCancel). Mainnet: 653713 XRPCAT taken from rfD9A7M8's 01EA78BD, and rn2zm7j6's unfunded 1F26E638 AND 8BB6A1F0 — on two different levels, the second behind rfD9A7M8's live 9EABC3F5 — deleted. Ours (F114): 1F26E638 reaped, 8BB6A1F0 kept; the live shadow then carried it through the ledger — present-vs-deleted, the book page 32 bytes long, …

- **fix(ledger): judge the taker's limit by the level's quality, not the offer's own rate (finding 116)** (`67140df`, xrpl-validator) #106732842 7BD990E595DF — rKv64MDJ buys 19.718743 "666" for 5.783103 XRP. Mainnet crosses rJ4EpEPT's underfunded 16DAB624 for its whole 12.709 "666" (3727433 drops), deletes the offer and its owner page, and rests the taker's 7.00924955970754 remainder. Ours crossed nothing and rested the full offer: six maker-side nodes missing from our write-set, the taker's root and resting offer off.
- **fix(ledger): a level this apply emptied is stepped through, not reported live (finding 117)** (`b0e848e`, xrpl-validator) #106733197 DFD1E132035C — rpK3DLr8 buys 3111.618064 XRP for RLUSD (tfImmediateOrCancel). Mainnet takes three underfunded bot makers whole on three levels (0.8, 0.6, 0.4 XRP) and deletes six more of their offers with their pages: the dead ones behind each fill and rD7q6194Bw's remaining D8118C65 and 119270E3, two levels further on. Ours took the same three fills and left those two behind — the maker's OwnerCount one …
- **fix(ledger): the first book hop composes its in-side transfer rate into the strand's quality function (finding 119)** (`44348d2`, xrpl-validator) #106732759 34694521561A — a self-payment of 16.45 UNI for SendMax 100.32264774 USDT, tfPartialPayment|tfLimitQuality, six paths, two of them through AMM pools (USDT/XRPS, XRPS/UNI). Mainnet delivers 15.62 UNI; ours 5.83 — six lines off, the two pools' balances among them.
- **fix(ledger): the remaining-in budget is an STAmount (finding 118)** (`f852b6b`, xrpl-validator) #106733048 E4014705 — a tfSell of 8267788.778091 CSC for LHT: 109292.827 CSC at the first level, the rest at a 1:1 maker. Mainnet credits the taker 8267867.899868732 LHT; ours 8267867.899868731. #106732165 185A4B50 — a tfSell|tfIoC of 200 EVR under a 1.002 issuer rate: the maker's residual TakerPays is 332.7392751903707 on mainnet, 332.7392751903708 in ours.
- **fix(ledger): the pool turn anchors on the residual tip inside the inflated limit — the taker's own tip included (findings 120, 121)** (`e9f2d58`, xrpl-validator) Two faces of one rippled rule. `tryAMM(offers.tip().quality())` anchors the pool's offer on the RAW head of the book, and `BookOfferCrossingStep::qualityThreshold(lobQuality)` unanchors it (the limitOut-sized max offer) only when `qualityThreshold_ > lobQuality`, i.e. the tip is WORSE than the transfer-rate-inflated limit (BookStep.cpp). The head is whatever the stream lands on — the taker's own resting offer …
- **fix(ledger): the funds-exhausting fill takes the line's own balance; a bridged round's multiPath and self-tip removal follow rippled's pass (findings 122, 123; finding 121 refined)** (`4199029`, xrpl-validator) Finding 122 — #106734972 E7CB46461F09 and #106734622 C1183E11AEC3, two IoC sales that run the taker's line dry. rippled's last, funds-limited fill takes what the LINE still holds — the DirectStep's `maxSrcToDst`, the balance as the ledger carries it after every earlier iteration's 16-digit half-even debit — 653.9861023930549 after thirty-eight debits where the budget's sorted-fold remainder (finding 118) says …550, …
- **fix(ledger): the out-limited input is clamped to the underfunded maker's own input (finding 124)** (`6abd978`, xrpl-validator) #106735594 982E2CDBD610 — rapido pays 605278 drops for 2.065196669826816 "666" through rJ4EpEPT's offer, funded for 2.06519666982682 of its 20.021509. Mainnet takes 605277 drops; ours took 605278 — the maker's root and residual offer one drop off.
- **fix(ledger): a bridged round ranks the direct strand on the book's live tip, not the emptied level under the cursor (finding 125)** (`fdb2592`, xrpl-validator) rippled's `activateNext` sorts the strands on `qualityUpperBound` — `tipOfferQuality`, a `BookTip::step` over the view as it stands when the iteration opens: the winning strand's sandbox and every stream's dead offers were applied at the END of the previous iteration (StrandFlow.h, `best->sb.apply(sb)` and the `ofrsToRm` `offerDelete` loop), and `BookTip` reads the directory, not the offer, so a self-offer or an …

- **fix(ledger): the mixed engine's book segment nets its carry at mulRatio-nearest (finding 126)** (`b6f8c45`, xrpl-validator) The fifth floor of the net-division family. rippled's `limitStepIn` nets the input a book step is fed at `mulRatio(stpAmt.in, QUALITY_ONE, trIn, false)` — IOUAmount's mulRatio: the quotient carried to eighteen digits, normalized to sixteen at Number's half-even nearest, and only then bumped by the roundUp flag. strand_pass's hop joint, spend0, the live-line clamp and the hop `avail` all moved to `mul_ratio` for …
- **fix(ledger): an XRP side of a maker's residual subtracts in exact drops (finding 127)** (`e7b7927`, xrpl-validator) `offer_residual` rounded every maker remainder back to sixteen significant digits — `TOffer::consume` subtracts through STAmount, and for an IOU that is Number's half-even nearest. For a native side it is not: STAmount holds drops in an int64 and `operator-` is exact.
- **fix(ledger): a bridged strand's limit-out function composes the tip's offer, not the forced pool (finding 128)** (`8dca547`, xrpl-validator) rippled's `limitOut` builds the strand's quality function step by step from `getQualityFunc` → `tipOfferQualityF` → `tip()`, and `tip()` hands back the pool's offer only when the offer it would GENERATE beats the book's tip on its own quality (`ammOffer->quality() > lobQuality`, BookStep.cpp). The single-path forcing — `qualityThreshold` returning nullopt when the strand's limit reads better than the leg's tip, a …
- **fix(ledger): an encoded rate rounds to nearest after the legacy half-up digit; the fib seed multiplies at nearest (findings 129, 130)** (`66db675`, xrpl-validator) Two rounding rules in `amm_swap.rs`, each with its own vectors, committed together because they share the file and the same drill set proved them.
- **fix(ledger): a DepositPreauth entry carries Flags and its owner-directory node (finding 132)** (`fc91ef2`, xrpl-validator) rippled's `DepositPreauth::doApply` builds the entry, inserts its key into the owner directory and records the page in `sfOwnerNode`; every ledger entry carries `sfFlags`. The transactor here was the early simplified one: it wrote the object bare — no Flags, no OwnerNode — and never touched the directory, and it answered an existing authorization with success where rippled's preclaim says tecDUPLICATE (and …
- **fix(ledger): a forward-driven pool turn swaps the carried input and clamps only the delivery (finding 131)** (`ad7b3eb`, xrpl-validator) rippled sizes a strand backwards and, once a step is limited — SendMax at the source, or a step that cannot supply its request — resets the sandbox and runs every later step forwards, IN-driven: `BookStep::fwdImp` consumes offers by input, and a single-path pool re-swaps the carried input (`limitStepIn` → `AMMOffer::limitIn` → `swapAssetIn`). The reverse cache reinstates the reverse amounts only when …
- **fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)** (`7a5e5f4`, xrpl-validator) `BookStep::fwdImp`'s reconciliation: when the forward output exceeds the reverse cache, `limitStepOut(cache.out)` re-sizes the offer to the reverse amounts, and if THAT step input — the reverse net re-grossed by the in-side transfer rate — equals the forward input as carried, the reverse amounts stand and the whole gross is consumed; the fee absorbs the sliver. Finding 131 swept every tiny sliver into a single …
- **fix(ledger): a book pass takes the whole same-quality group; pool slices price by their 16-digit rate (findings 137, 138)** (`5879b9c`, xrpl-validator) Finding 137 — rippled's `BookStep::forEachOffer` consumes, in ONE pass, every offer whose quality equals the first offer's (`else if (*ofrQ != offer.quality()) return false;`) — the whole same-quality directory group — and the flow iterates only when the quality changes. Ours took one leg-B offer per iteration. The pool on leg A advances its Fibonacci slice per ITERATION (`AMMContext::curIters`), so a ladder of …
- **fix(ledger): SignerListSet replace re-inserts the owner-directory entry (finding 136)** (`0fd2a90`, xrpl-validator) rippled's `replaceSignerList` (SignerListSet.cpp:299-355) is remove-then-create, for a fresh list and a replacement alike: `removeSignersFromLedger` = dirRemove at the old list's OwnerNode hint + OwnerCount -1

(lsfOneOwnerCount; a legacy list without the flag gives back 2 + n) + erase; then `checkReserve(preFeeBalance_, ownerCountDelta = 1)`; then a fresh SLE, dirInsert — the key lands at the END of the owner …

- **fix(ledger): reserve gate applies to crossed offers on the pre-fee balance (finding 135)** (`c15f3b6` , xrpl-validator) rippled OfferCreate.cpp:835-851 runs the reserve check after crossing, unconditionally: `preFeeBalance_` (the creator's balance before this transaction's fee, Transactor.cpp:896) against the reserve for one more object at the CURRENT owner count. Below it the remainder is never placed — tecINSUF_RESERVE_OFFER when nothing crossed, tesSUCCESS with the fills kept and the residual dropped when something did ("If …
- **fix(ledger): the taker's spend is capped by the deferred-credits balance; a maker's XRP reserve holds at the flow's original owner count (findings 133, 134)** (`be1a76d` , xrpl-validator) Two faces of rippled's PaymentSandbox, committed together because they share offer.rs and one drill set proved them.
- **fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)** (`4d86d53` , xrpl-validator) `BookStep::fwdImp` 's reconciliation: when the forward output exceeds the reverse cache, `limitStepOut(cache.out)` re-sizes the offer to the reverse amounts, and if THAT step input — the reverse net re-grossed by the in-side transfer rate — equals the forward input as carried, the reverse amounts stand and the whole gross is consumed; the fee absorbs the sliver. Finding 131 swept every tiny sliver into a single …
- **fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)** (`0ff3de8` , xrpl-validator) `BookStep::fwdImp` 's reconciliation: when the forward output exceeds the reverse cache, `limitStepOut(cache.out)` re-sizes the offer to the reverse amounts, and if THAT step input — the reverse net re-grossed by the in-side transfer rate — equals the forward input as carried, the reverse amounts stand and the whole gross is consumed; the fee absorbs the sliver. Finding 131 swept every tiny sliver into a single …
- **fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)** (`81f8f89` , xrpl-validator) `BookStep::fwdImp` 's reconciliation: when the forward output exceeds the reverse cache, `limitStepOut(cache.out)` re-sizes the offer to the reverse amounts, and if THAT step input — the reverse net re-grossed by the in-side transfer rate — equals the forward input as carried, the reverse amounts stand and the whole gross is consumed; the fee absorbs the sliver. Finding 131 swept every tiny sliver into a single …
- **fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)** (`cb0ade1` , xrpl-validator) `BookStep::fwdImp` 's reconciliation: when the forward output exceeds the reverse cache, `limitStepOut(cache.out)` re-sizes the offer to the reverse amounts, and if THAT step input — the reverse net re-grossed by the in-side transfer rate — equals the forward input as carried, the reverse amounts stand and the whole gross is consumed; the fee absorbs the sliver. Finding 131 swept every tiny sliver into a single …
- **fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)** (`98ba356` , xrpl-validator) `BookStep::fwdImp` 's reconciliation: when the forward output exceeds the reverse cache, `limitStepOut(cache.out)` re-sizes the offer to the reverse amounts, and if THAT step input — the reverse net re-grossed by the in-side transfer rate — equals the forward input as carried, the reverse amounts stand and the whole gross is consumed; the fee absorbs the sliver. Finding 131 swept every tiny sliver into a single …

- **fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)** (`c6deec0` , xrpl-validator) `BookStep::fwdImp` 's reconciliation: when the forward output exceeds the reverse cache, `limitStepOut(cache.out)` re-sizes the offer to the reverse amounts, and if THAT step input — the reverse net re-grossed by the in-side transfer rate — equals the forward input as carried, the reverse amounts stand and the whole gross is consumed; the fee absorbs the sliver. Finding 131 swept every tiny sliver into a single …
- **fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)** (`3dac6cd` , xrpl-validator) `BookStep::fwdImp` 's reconciliation: when the forward output exceeds the reverse cache, `limitStepOut(cache.out)` re-sizes the offer to the reverse amounts, and if THAT step input — the reverse net re-grossed by the in-side transfer rate — equals the forward input as carried, the reverse amounts stand and the whole gross is consumed; the fee absorbs the sliver. Finding 131 swept every tiny sliver into a single …
- **fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)** (`52b34c3` , xrpl-validator) `BookStep::fwdImp` 's reconciliation: when the forward output exceeds the reverse cache, `limitStepOut(cache.out)` re-sizes the offer to the reverse amounts, and if THAT step input — the reverse net re-grossed by the in-side transfer rate — equals the forward input as carried, the reverse amounts stand and the whole gross is consumed; the fee absorbs the sliver. Finding 131 swept every tiny sliver into a single …
- **fix(ledger): a book pass takes the whole same-quality group; pool slices price by their 16-digit rate (findings 137, 138)** (`a73e77b` , xrpl-validator) Finding 137 — rippled's `BookStep::forEachOffer` consumes, in ONE pass, every offer whose quality equals the first offer's (`else if (*ofrQ != offer.quality()) return false;`) — the whole same-quality directory group — and the flow iterates only when the quality changes. Ours took one leg-B offer per iteration. The pool on leg A advances its Fibonacci slice per ITERATION (`AMMContext::curIters`), so a ladder of …
- **fix(ledger): SignerListSet replace re-inserts the owner-directory entry (finding 136)** (`c266cff` , xrpl-validator) rippled's `replaceSignerList` (SignerListSet.cpp:299-355) is remove-then-create, for a fresh list and a replacement alike: `removeSignersFromLedger` = dirRemove at the old list's OwnerNode hint + OwnerCount -1 (lsfOneOwnerCount; a legacy list without the flag gives back 2 + n) + erase; then `checkReserve(preFeeBalance_, ownerCountDelta = 1)` ; then a fresh SLE, dirInsert — the key lands at the END of the owner …
- **fix(ledger): reserve gate applies to crossed offers on the pre-fee balance (finding 135)** (`4eed25f` , xrpl-validator) rippled OfferCreate.cpp:835-851 runs the reserve check after crossing, unconditionally: `preFeeBalance_` (the creator's balance before this transaction's fee, Transactor.cpp:896) against the reserve for one more object at the CURRENT owner count. Below it the remainder is never placed — tecINSUF_RESERVE_OFFER when nothing crossed, tesSUCCESS with the fills kept and the residual dropped when something did ("If …
- **fix(ledger): the taker's spend is capped by the deferred-credits balance; a maker's XRP reserve holds at the flow's original owner count (findings 133, 134)** (`1265494` , xrpl-validator) Two faces of rippled's PaymentSandbox, committed together because they share offer.rs and one drill set proved them.
- **fix(ledger): a book pass takes the whole same-quality group; pool slices price by their 16-digit rate (findings 137, 138)** (`75efdb1` , xrpl-validator) Finding 137 — rippled's `BookStep::forEachOffer` consumes, in ONE pass, every offer whose quality equals the first offer's (`else if (*ofrQ != offer.quality()) return false;`) — the whole same-quality directory group — and the flow iterates only when the quality changes. Ours

took one leg-B offer per iteration. The pool on leg A advances its Fibonacci slice per ITERATION (`AMMContext::curIters`), so a ladder of …

- **fix(ledger): a book pass takes the whole same-quality group; pool slices price by their 16-digit rate (findings 137, 138)** (`c8774b1`, xrpl-validator) Finding 137 — rippled's `BookStep::forEachOffer` consumes, in ONE pass, every offer whose quality equals the first offer's (`else if (*ofrQ != offer.quality()) return false;`) — the whole same-quality directory group — and the flow iterates only when the quality changes. Ours took one leg-B offer per iteration. The pool on leg A advances its Fibonacci slice per ITERATION (`AMMContext::curIters`), so a ladder of …
- **fix(ledger): SignerListSet replace re-inserts the owner-directory entry (finding 136)** (`f6c5d75`, xrpl-validator) rippled's `replaceSignerList` (SignerListSet.cpp:299-355) is remove-then-create, for a fresh list and a replacement alike: `removeSignersFromLedger` = dirRemove at the old list's OwnerNode hint + OwnerCount -1 (lsfOneOwnerCount; a legacy list without the flag gives back 2 + n) + erase; then `checkReserve(preFeeBalance_, ownerCountDelta = 1)`; then a fresh SLE, dirInsert — the key lands at the END of the owner …
- **fix(ledger): reserve gate applies to crossed offers on the pre-fee balance (finding 135)** (`e7b4239`, xrpl-validator) rippled OfferCreate.cpp:835-851 runs the reserve check after crossing, unconditionally: `preFeeBalance_` (the creator's balance before this transaction's fee, Transactor.cpp:896) against the reserve for one more object at the CURRENT owner count. Below it the remainder is never placed — tecINSUF_RESERVE_OFFER when nothing crossed, tesSUCCESS with the fills kept and the residual dropped when something did ("If …
- **fix(ledger): the taker's spend is capped by the deferred-credits balance; a maker's XRP reserve holds at the flow's original owner count (findings 133, 134)** (`b28172a`, xrpl-validator) Two faces of rippled's PaymentSandbox, committed together because they share offer.rs and one drill set proved them.
- **fix(ledger): strands are admitted against the fee-inclusive limit; self-offers leave the book only when the direct strand executes (finding 139)** (`168f252`, xrpl-validator) rippled has ONE limitQuality for the crossing — `Quality{takerAmount.out, sendMax}` with the gateway transfer fee INSIDE sendMax (OfferCreate.cpp: 380-393) — and holds the strands' RAW upper bounds against it (`adjustQualityWithFees` returns the tip's own quality for CLOB and multi-path pool offers): strand activation (StrandFlow.h `activateNext`), the per-strand `qualityUpperBound < limitQuality` skip, and the …
- **fix(ledger): a path that only names the SendMax issuer at its head is the default strand (finding 140)** (`2399f83`, xrpl-validator) rippled's `toStrands` inserts a strand only if it is not already in the list (`hasStrand`, PaySteps.cpp). A named path `[rMxCK (the SendMax issuer), XRP]` normalises to exactly the default strand `[src → issuer, RLUSD/XRP book]` — the head hop src → SendMax issuer is on every strand — so the two collapse to ONE strand and the flow runs single-path.
- **fix(ledger): an unanchored pool tips its leg by SPOT, and the single pass is sized to the limit (finding 141)** (`a4080bb`, xrpl-validator) `AMMLiquidity::maxOffer` — the offer a single-path leg gets when its book tip is worse than the strand's limit (`qualityThreshold` returns nullopt, no anchor) — carries `Quality{balances}`, the pool's feeless SPOT, as its quality: `AMMOffer(*this, {swapAssetOut(balances, out), out}, balances, Quality{balances})`. The tip compare that follows, `ammOffer->quality() > lobQuality`, is therefore spot against the book, …
- **fix(ledger): iteration 0 takes multiPath from the strands activateNext keeps (finding 142)** (`03e305a`, xrpl-validator) `flow()` (StrandFlow.h) runs `activateNext(sb, limitQuality)` at the TOP of every iteration — the first included — and only then `ammContext.setMultiPath(activeStrands.size() > 1)`: a strand whose

upper bound misses limitQuality is dropped before any pool is asked for an offer. Ours handed round 0 the built-strand entry (`multi_prev`, "strands.size() > 1"), which is what `Flow.cpp` sets before the loop and what the …

- **fix(ledger): the crossing's residual is the original less the STAmount sum of the iterations' fills (finding 144)** (`2155052`, xrpl-validator) `flow()` accumulates each iteration's actual in and out in `savedIns` / `savedOuts`, and `OfferCreate::flowCross` derives the residual from `sum(savedIns)` (tfSell: `afterCross.in -= divideRound(actualIn, xferRate, up)`) or `sum(savedOuts)` (a buy: `afterCross.out -= actualOut`), the counterpart then priced through the original rate (OfferCreate.cpp:497-520). The sum canonicalizes at every `+=`; a running …
- **fix(ledger): the direct pool is consulted in a single-path round only if its strand survived activation (finding 145)** (`6e3466f`, xrpl-validator) `activateNext` (StrandFlow.h) keeps a strand only if its upper bound meets limitQuality, and that bound is composed BEFORE the round's `setMultiPath`: with the previous iteration's multiPath, the direct book's tip is the pool's Fibonacci slice at the current iteration count. Only a kept strand is flowed; a single-path round then asks the pool for its anchored (or limit-sized) offer.
- **fix(ledger): a strand's first pool hop is sized by the reverse pass, not swept to the SendMax (finding 147)** (`47905e9`, xrpl-validator) The tiny-sliver sweep in `consume` (finding 131) models `BookStep::fwdImp` consuming the WHOLE product a previous step handed it: when the input's unspent remainder is under one part in a billion, the pool takes it all and the overshoot flows on. A strand's FIRST book step is not fed by a product. Its forward input is the reverse pass's own answer (`limitStepOut` → `swapAssetOut(want)`), already the exact in, …
- **fix(ledger): re-run a limiting book level and re-price its last maker (finding 149)** (`e801434`, xrpl-validator) rippled's strand driver treats a book step that could not satisfy the requested output as LIMITING and re-executes it in reverse with the step's own result as the target (StrandFlow.h:176-184, `stepOut = r.second`). BookStep::revImp then walks the same offers a second time with `remainingOut = out - sum(savedOuts)` after each, and the level's last offer is trimmed by `limitStepOut` to that remainder instead of its …
- **fix(ledger): record a credential's directory pages (finding 150)** (`bafc30c`, xrpl-validator) CredentialCreate stores the page of each owner directory the credential is inserted into — `sleCred->setFieldU64(sfIssuerNode, *page)` after the issuer's dirInsert and `sfSubjectNode` after the subject's, none of the latter on a self-issued credential (CredentialCreate.cpp doApply). Both are UInt64 fields with two-byte headers, ten bytes each; our object carried neither, so every credential we created was 20 bytes …
- **fix(ledger): a strand that flows nothing never returns (finding 151)** (`802b6a7`, xrpl-validator) rippled's payment driver keeps its candidate strands in `ActiveStrands`: `next_` is seeded with every strand, and each iteration `activateNext` turns it into the sorted active set, sorting and filtering (no bound, or a bound past limitQuality) only when more than one strand is pending — a lone pending strand is activated unexamined. The iteration then pushes back only the strand that produced (unless it went …
- **fix(ledger): an expired check cannot be cashed or created (finding 152)** (`02653b5`, xrpl-validator) rippled judges a check's Expiration against the parent close time — `hasExpired(view, exp)` is `parentCloseTime >= *exp` — and returns tecEXPIRED from CheckCash::preclaim (CheckCash.cpp:132-136), after the destination check and before any funds move; CheckCreate::preclaim applies the same rule to a check created already expired (CheckCreate.cpp:179-183). We modelled neither.
- **fix(ledger): the direct pass removes a self-offer only inside the limit, and stops at one beyond it (finding 143)** (`399e5b4`, xrpl-validator) `limitSelfCrossQuality` (BookStep.cpp) deletes the taker's own offer off the direct tip only when `offer.quality() >= qualityThreshold_`; one beyond the limit falls through to `checkQualityThreshold`, which fails, and the pass ENDS there. Finding 139 moved the reap onto the direct

strand's own walk and, in doing so, dropped the quality gate the round-opening reap had carried: `live_head(mutate_self)` deleted every …

- **fix(ledger): an offer that became unfunded during the crossing is not removed for good (finding 153)** (`18ff76e`, xrpl-validator) rippled's offer stream deletes every offer it steps past in the iteration's sandbox (BookTip::step removes the previous entry), but only an offer that was ALREADY unfunded when the iteration began is added to the permanent-removal set: OfferStream::step compares the owner's funds in the cancel view with the funds now and, when they differ, logs "became unfunded" without `permRmOffer` (OfferStream.cpp:279-304). The …
- **fix(ledger): a stranger may burn an NFToken only if it is burnable and they are its issuer or minter (finding 154)** (`8405e57`, xrpl-validator) NFTokenBurn::preclaim lets a token's owner burn it unconditionally; any other account needs the token flagged burnable (0x0001 in the NFTokenID's flag word) and must be its issuer or the issuer's current NFTokenMinter — an issuer with no minter set fails that comparison, an issuer whose root is gone skips it (NFTokenBurn.cpp preclaim). We had no permission check at all: every burn with an Owner field went through.

Tests and campaigns

- **test(node): bundles rebuild in-ledger objects exactly and can pin untouched targets** (`0bbd06e`, xrpl-validator)

Independence program

No independence record for UTC day 2026-09-03; records begin 2026-10-20.

Network

No ledger records for this day: the reference node holds ledgers from 2026-10-04 22:52 UTC (#107,436,647).

Amendment watch

No amendment records for this day.

Open items

- Receipt R-01 (#106,727,063): open, under investigation.
- Receipt R-02 (#106,727,070): open, under investigation.
- Receipt R-03 (#106,727,083): open, under investigation.
- Receipt R-05 (#106,727,098): open, under investigation.
- Receipt R-06 (#106,727,665): open, under investigation.
- Receipt R-09 (#106,731,077): open, under investigation.
- Receipt R-16 (#106,733,194): open, under investigation.

- Receipt R-23 (#106,734,469): open, under investigation.
- Receipt R-31 (#106,735,617): open, under investigation.
- Receipt R-32 (#106,735,862): open, under investigation.
- Receipt R-33 (#106,735,879): open, under investigation.
- Receipt R-34 (#106,735,893): open, under investigation.
- Receipt R-35 (#106,735,894): open, under investigation.
- Receipt R-36 (#106,735,895): open, under investigation.
- Receipt R-37 (#106,735,896): open, under investigation.
- Receipt R-38 (#106,735,916): open, under investigation.
- Receipt R-39 (#106,735,981): open, under investigation.
- Receipt R-42 (#106,735,990): open, under investigation.
- Receipt R-43 (#106,735,991): open, under investigation.
- Receipt R-44 (#106,735,994): open, under investigation.
- Receipt R-45 (#106,736,013): open, under investigation.
- Receipt R-46 (#106,736,016): open, under investigation.
- Receipt R-47 (#106,736,065): open, under investigation.
- Receipt R-48 (#106,736,232): open, under investigation.
- Receipt R-49 (#106,736,287): open, under investigation.
- Receipt R-50 (#106,736,550): open, under investigation.
- Receipt R-55 (#106,736,692): open, under investigation.
- Receipt R-56 (#106,736,876): open, under investigation.
- Receipt R-57 (#106,736,944): open, under investigation.
- Receipt R-58 (#106,736,948): open, under investigation.
- Receipt R-59 (#106,737,459): open, under investigation.
- Receipt R-65 (#106,738,977): open, under investigation.
- Receipt R-67 (#106,739,154): open, under investigation.
- Receipt R-73 (#106,741,232): open, under investigation.
- Receipt R-74 (#106,741,657): open, under investigation.
- Receipt R-77 (#106,742,138): open, under investigation.
- Receipt R-80 (#106,742,536): open, under investigation.
- Receipt R-81 (#106,742,566): open, under investigation.
- Receipt R-82 (#106,742,761): open, under investigation.
- Receipt R-83 (#106,742,891): open, under investigation.
- Receipt R-86 (#106,743,257): open, under investigation.
- Receipt R-87 (#106,743,342): open, under investigation.
- Receipt R-95 (#106,745,732): open, under investigation.
- Receipt R-96 (#106,745,757): open, under investigation.
- Receipt R-100 (#106,747,216): open, under investigation.

Corrections to earlier reports

None.

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **MPC Threshold-Signature Policy Gate** (custody) Add a new `AccountPolicy` ledger object (ltACCOUNT_POLICY, owner-reserved, keylet::accountPolicy(account)) set via a `PolicySet` transaction, carrying
- **Oracle-Triggered Stablecoin Circuit Breaker** (tokens) Add a circuit-breaker config to `MPTokenIssuance` — sfReferenceOracles (a bounded STVector256 of up to 8 PriceOracle keylets) and sfDepegThresholdBPS
- **Weighted Basket Vaults for Index Funds** (lending) Add a `tfBasket` flag to `VaultCreate` and a bounded sfBasketAssets STArray (max 10 {Asset, WeightBPS} entries summing to 10000) stored on the Vault S

Appendix — commits

Committed	Merged into main	Repo	Commit	Subject
09-03 05:22	09-03 05:22	xrpl-validator	`32512b0`	fix(ledger): single-path bridge legs take and judge the pool offer rippled's tip() hands back (findings 103, 111)
09-03 06:22	09-03 06:22	xrpl-validator	`9534ff1`	fix(ledger): a dust offer whose quality rounds worse is removed, not crossed (finding 112)
09-03 09:16	09-03 09:16	xrpl-validator	`029a608`	fix(ledger): erasing a sender's in-flight line removes its entries, not the pages (finding 113)
09-03 10:33	09-03 10:33	xrpl-validator	`f55ce32`	fix(ledger): killed crossings still reap what the rev pass stepped over (finding 114)
09-03 11:26	09-03 11:26	xrpl-validator	`f7156c9`	fix(ledger): the rev pass steps past a maker it emptied and reaps behind it (finding 115)
09-03 12:05	09-03 12:05	xrpl-validator	`67140df`	fix(ledger): judge the taker's limit by the level's quality, not the offer's own rate (finding 116)
09-03 12:15	09-03 12:15	xrpl-validator	`b0e848e`	fix(ledger): a level this apply emptied is stepped through, not reported live (finding 117)
09-03 13:13	09-03 13:13	xrpl-validator	`44348d2`	fix(ledger): the first book hop composes its in-side transfer rate into the strand's quality function (finding 119)
09-03 13:13	09-03 13:13	xrpl-validator	`f852b6b`	fix(ledger): the remaining-in budget is an STAmount (finding 118)
09-03 14:00	09-03 14:00	xrpl-validator	`e9f2d58`	fix(ledger): the pool turn anchors on the residual tip inside the inflated limit — the taker's own tip included (findings 120, 121)
09-03 14:45	09-03 14:45	xrpl-validator	`4199029`	fix(ledger): the funds-exhausting fill takes the line's own balance; a bridged round's multiPath and self-tip removal follow rippled's pass (findings 122, 123; finding 121 refined)
09-03 14:53	09-03 14:53	xrpl-validator	`6abd978`	fix(ledger): the out-limited input is clamped to the underfunded maker's own input (finding 124)

Committed	Merged into main	Repo	Commit	Subject
09-03 15:37	09-03 15:37	xrpl-validator	`fdb2592`	fix(ledger): a bridged round ranks the direct strand on the book's live tip, not the emptied level under the cursor (finding 125)
09-03 15:37	09-03 15:37	xrpl-validator	`b6f8c45`	fix(ledger): the mixed engine's book segment nets its carry at mulRatio-nearest (finding 126)
09-03 16:02	09-03 16:02	xrpl-validator	`e7b7927`	fix(ledger): an XRP side of a maker's residual subtracts in exact drops (finding 127)
09-03 16:15	09-03 16:15	xrpl-validator	`8dca547`	fix(ledger): a bridged strand's limit-out function composes the tip's offer, not the forced pool (finding 128)
09-03 16:25	09-03 16:25	xrpl-validator	`66db675`	fix(ledger): an encoded rate rounds to nearest after the legacy half-up digit; the fib seed multiplies at nearest (findings 129, 130)
09-03 17:18	09-03 17:18	xrpl-validator	`fc91ef2`	fix(ledger): a DepositPreauth entry carries Flags and its owner-directory node (finding 132)
09-03 17:18	09-03 17:18	xrpl-validator	`ad7b3eb`	fix(ledger): a forward-driven pool turn swaps the carried input and clamps only the delivery (finding 131)
09-03 19:30	09-03 19:30	xrpl-validator	`7a5e5f4`	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)
09-03 19:31	09-03 19:31	xrpl-validator	`5879b9c`	fix(ledger): a book pass takes the whole same-quality group; pool slices price by their 16-digit rate (findings 137, 138)
09-03 19:31	09-03 19:31	xrpl-validator	`0fd2a90`	fix(ledger): SignerListSet replace re-inserts the owner-directory entry (finding 136)
09-03 19:31	09-03 19:31	xrpl-validator	`c15f3b6`	fix(ledger): reserve gate applies to crossed offers on the pre-fee balance (finding 135)
09-03 19:31	09-03 19:31	xrpl-validator	`be1a76d`	fix(ledger): the taker's spend is capped by the deferred-credits balance; a maker's XRP reserve holds at the flow's original owner count (findings 133, 134)
09-03 19:32	09-03 19:32	xrpl-validator	`4d86d53`	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)
09-03 19:33	09-03 19:33	xrpl-validator	`0ff3de8`	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)
09-03 19:33	09-03 19:33	xrpl-validator	`81f8f89`	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)
09-03 19:33	09-03 19:33	xrpl-validator	`cb0ade1`	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)
09-03 19:33	09-03 19:33	xrpl-validator	`98ba356`	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)
09-03 19:33	09-03 19:33	xrpl-validator	`c6deec0`	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)
09-03 19:33	09-03 19:33	xrpl-validator	`3dac6cd`	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)
09-03 19:33	09-03 19:33	xrpl-validator	`52b34c3`	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)
09-03 19:33	09-03 19:33	xrpl-validator	`a73e77b`	fix(ledger): a book pass takes the whole same-quality group; pool slices price by their 16-digit rate (findings 137, 138)
09-03 19:33	09-03 19:33	xrpl-validator	`c266cff`	fix(ledger): SignerListSet replace re-inserts the owner-directory entry (finding 136)
09-03 19:33	09-03 19:33	xrpl-validator	`4eed25f`	fix(ledger): reserve gate applies to crossed offers on the pre-fee balance (finding 135)
09-03 19:33	09-03 19:33	xrpl-validator	`1265494`	fix(ledger): the taker's spend is capped by the deferred-credits balance; a maker's XRP reserve holds at the flow's original owner count (findings 133, 134)
09-03 19:34	09-03 19:34	xrpl-validator	`75efdb1`	fix(ledger): a book pass takes the whole same-quality group; pool slices price by their 16-digit rate (findings 137, 138)

Committed	Merged into main	Repo	Commit	Subject
09-03 19:48	09-03 19:48	xrpl-validator	`c8774b1`	fix(ledger): a book pass takes the whole same-quality group; pool slices price by their 16-digit rate (findings 137, 138)
09-03 19:48	09-03 19:48	xrpl-validator	`f6c5d75`	fix(ledger): SignerListSet replace re-inserts the owner-directory entry (finding 136)
09-03 19:48	09-03 19:48	xrpl-validator	`e7b4239`	fix(ledger): reserve gate applies to crossed offers on the pre-fee balance (finding 135)
09-03 19:48	09-03 19:48	xrpl-validator	`b28172a`	fix(ledger): the taker's spend is capped by the deferred-credits balance; a maker's XRP reserve holds at the flow's original owner count (findings 133, 134)
09-03 19:54	09-03 19:54	xrpl-validator	`168f252`	fix(ledger): strands are admitted against the fee-inclusive limit; self-offers leave the book only when the direct strand executes (finding 139)
09-03 19:56	09-03 19:56	xrpl-validator	`2399f83`	fix(ledger): a path that only names the SendMax issuer at its head is the default strand (finding 140)
09-03 20:09	09-03 20:09	xrpl-validator	`a4080bb`	fix(ledger): an unanchored pool tips its leg by SPOT, and the single pass is sized to the limit (finding 141)
09-03 21:35	09-03 21:35	xrpl-validator	`03e305a`	fix(ledger): iteration 0 takes multiPath from the strands activateNext keeps (finding 142)
09-03 22:12	09-03 22:12	xrpl-validator	`2155052`	fix(ledger): the crossing's residual is the original less the STAmount sum of the iterations' fills (finding 144)
09-03 22:12	09-03 22:12	xrpl-validator	`6e3466f`	fix(ledger): the direct pool is consulted in a single-path round only if its strand survived activation (finding 145)
09-03 22:47	09-03 22:47	xrpl-validator	`47905e9`	fix(ledger): a strand's first pool hop is sized by the reverse pass, not swept to the SendMax (finding 147)
09-04 00:16	09-04 00:16	xrpl-validator	`e801434`	fix(ledger): re-run a limiting book level and re-price its last maker (finding 149)
09-04 00:21	09-04 00:21	xrpl-validator	`bafc30c`	fix(ledger): record a credential's directory pages (finding 150)
09-04 00:42	09-04 01:11	xrpl-validator	`802b6a7`	fix(ledger): a strand that flows nothing never returns (finding 151)
09-04 00:44	09-04 01:11	xrpl-validator	`02653b5`	fix(ledger): an expired check cannot be cashed or created (finding 152)
09-04 02:26	09-04 02:26	xrpl-validator	`399e5b4`	fix(ledger): the direct pass removes a self-offer only inside the limit, and stops at one beyond it (finding 143)
09-04 02:41	09-04 02:41	xrpl-validator	`0bbd06e`	test(node): bundles rebuild in-ledger objects exactly and can pin untouched targets
09-04 03:33	09-04 03:33	xrpl-validator	`18ff76e`	fix(ledger): an offer that became unfunded during the crossing is not removed for good (finding 153)
09-04 03:54	—	xrpl-validator	`8405e57`	fix(ledger): a stranger may burn an NFToken only if it is burnable and they are its issuer or minter (finding 154)

Times in UTC. A commit appears on the day it was committed to main or merged into it.